Reproducing GTTA on BFCLv3: What I Saw

Natnael Bekele

natnaelbekele142@gmail.com · nathybekele.github.io · nathyBekele · LinkedIn · SA Inspector (live)

SUMMARY

1. **Dynamics Grounding (DG) works as reported:** +3.5 to +4.0 points in all three of my runs, vs. +3.5 in the paper.
2. **Syntactic Alignment (SA) didn't change how many tasks were solved**, at the paper's default or best setting.
3. **A possible reason:** in the released code, SA's step gets smaller as the prompt gets longer.

1 Goal

I wanted to understand *Test-Time Adaptation for LLM Agents via Environment Interaction* (GTTA; Chen et al., ICLR 2026) by rerunning it. It proposes two ways for an agent to adapt to a new environment. **Syntactic Alignment (SA)** learns a small vector δ added to the model's last hidden layer, updated with a few Adam steps on the current prompt. **Dynamics Grounding (DG)** lets the agent explore, turns what it saw into written rules, and adds them to the prompt.

One observation guided the rest: the logits are $(H+\delta)W^T = HW^T + \delta W^T$, so SA amounts to a fixed score added to every word in the vocabulary. That means I can read off exactly which words it pushes up.

2 Setup

- **Model and data:** Qwen2.5-14B-Instruct (bf16) on BFCLv3 multi-turn, the 200 tasks in the authors' repository (commit `24ba304`), scored with BFCL's own checker.
- **SA:** reimplemented from the released `vllm/model_runner.py` (a line-by-line port of the released gradient matches mine), with the same schedule: reset at each user turn, Adam steps on the whole prompt at every model call.
- **DG:** the released prompts and exploration goals, but Qwen-14B itself explores, writes and filters the rules (the paper used GPT-4.1/GPT-4o/o3).
- **Compute:** rented cloud GPUs (Modal), about $13 for all experiments.

3 Findings

DG works as reported. Without adaptation the agent solves 17.5–18.5% of tasks; with DG rules, 21.0–22.0% (Figure 1). Two of my runs hit the paper's numbers exactly. Identical reruns move scores by about 1 point, so the gain is bigger than the noise.

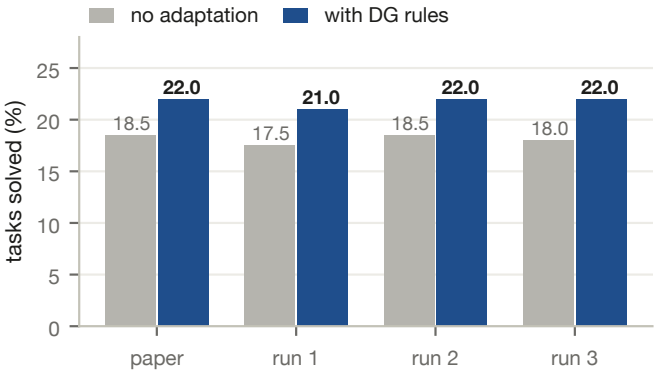


Figure 1. Tasks solved with and without DG rules, on all 200 tasks. My three runs use identical prompts.

SA didn't change how many tasks were solved. At the paper's default setting, SA changed at least one generated word in 53 of 200 tasks but never changed an outcome. At the paper's best setting for this model, it changed almost every run, with 5 extra wins and 4 extra losses (+0.5 points, 95% CI –2.5 to +3.5; Figure 2).

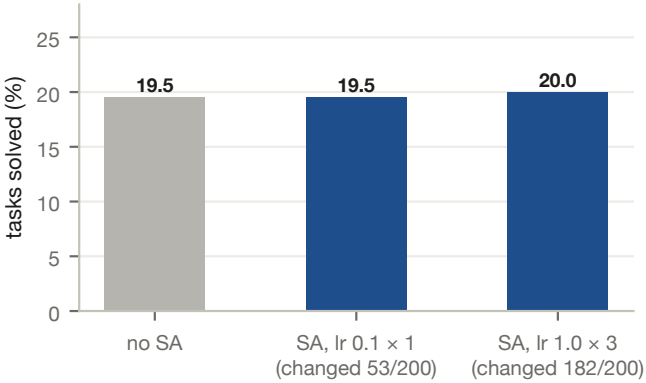


Figure 2. Tasks solved with SA inside the agent loop (released gradient), compared with no SA in the same loop. "Changed" counts tasks where SA altered at least one generated word.

A possible reason: the step shrinks as the prompt grows. The released code averages the gradient over the prompt twice (divide by the number of tokens, then `.mean(0)` over positions), which equals the exact gradient divided by the prompt length. On BFCL's 4K–8K-token prompts every coordinate ends up below Adam's $\varepsilon = 10^{-6}$, so the step is about 64× smaller than Adam would normally take, and it keeps

shrinking with length (Figure 3). The 1.5B model shows the same thing.

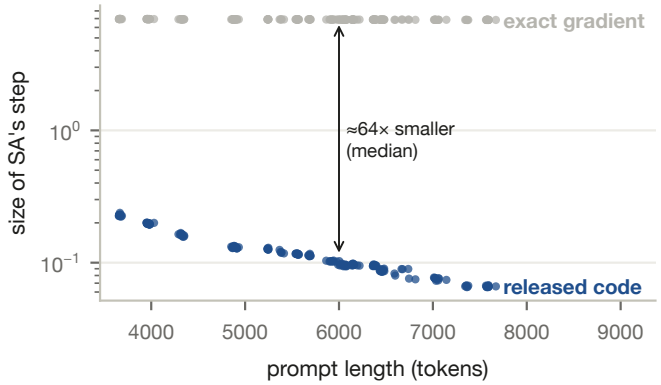


Figure 3. Size of SA's first step (‖δ‖, lr 0.1) on each of the 200 prompts, with the released code and with the exact gradient.

What SA pushes up. Its top words are the prompt's most frequent text: punctuation, newlines, and "Note", because BFCL appends "Note that the provided function is in Python 3 syntax" to every function (Figure 4). Only about 8% of the top words appear in the correct answer, and SA doesn't make the correct first call more likely at either setting (e.g. −0.0027 nats per token, 95% CI −0.0077 to +0.0019, at the best setting).

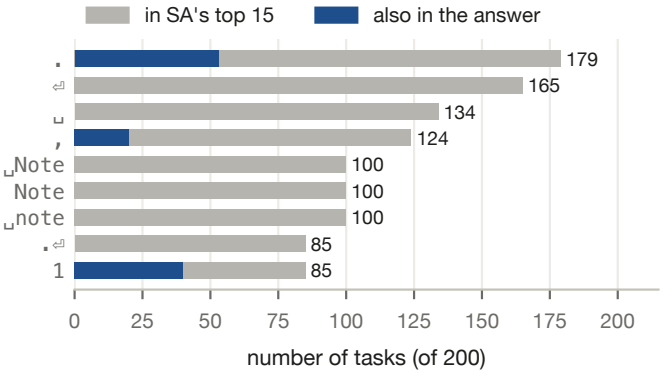


Figure 4. Words most often among SA's 15 most-boosted, across the 200 tasks, for a full-size step. The released code's tiny step leans the same way, punctuation first.

SA together with DG. DG's rules make prompts 36% longer, which shrinks SA's step further (median ‖δ‖ 0.108 → 0.070). This may be one reason the paper's simple SA+DG combination (21.0%) didn't beat DG alone (22.0%).

4 Process

1. I first measured how likely the model finds the correct first call, with and without SA. This is cheap: one forward pass covers every setting.
2. The step looked tiny, which led me to the double averaging. To be fair, I reran at the paper's best setting (lr 1.0, 3 steps); the step grew, but the correct call still didn't get more likely.
3. The first DG attempt failed silently: Qwen often writes several calls per step, BFCL rejected them, and 55 of 80 exploration logs came out empty. Running the first valid call (during exploration only) fixed it, giving 133 rules.
4. Scores moved by about 1 point between identical runs, so I repeated the DG runs before trusting a difference.
5. Last, I ran SA inside the agent loop. A task where SA never changes a generated word must end the same way, so I only reran the tasks it did change.
6. I corrected one inconsistency of my own: some medians had been computed on the 160 test tasks only. Every number here uses all 200 tasks.

5 Limitations

- I ran SA in my own PyTorch loop (one δ per task), not the original vLLM-based implementation. My loop scores 1–2 points higher overall, so I only compare SA with no SA inside the same loop.
- I tried one benchmark. BFCL prompts are mostly long function documentation, and WebArena may behave differently.
- For DG I used Qwen instead of GPT-4.1/GPT-4o/o3, plus the parsing change above.

6 Open questions

- Which user-turn masking setting (`MASK_TOKENS`) was used for the reported runs?
- Was the larger learning rate (1.0) meant to make up for the small step?

7 Reproducing these results

On the project site, the SA Inspector runs SA live on Qwen2.5-1.5B or 14B with either gradient, on a BFCL task or your own text, and the DG Inspector shows every DG rule and trajectory from the saved runs. Code, per-task results and a dated log of every decision are in the repository: github.com/nathyBekele/reproducing-gtta.

DG runs	`results/dg/dg-14b*/`
SA task success	`results/sa_agent/sa-agent-14b/`
Step size, top words	`results/summary/qwen2.5-14b-bf16*/`
Every decision	`notes/decisions.md`